R1.01 - Initiation au Développement : Variables, Mémoire RAM, Structures Conditionnelles et Répétitives

Thème principal : Algorithmique et Programmation Impérative

Synthèse exécutive : Ce cours magistral pose les fondations absolues de la programmation informatique pour les étudiants de BUT 1 Informatique à l'IUT de Bayonne et du Pays Basque (Campus d'Anglet). Il formalise la manière dont un ordinateur manipule la mémoire vive (RAM) à travers des variables typées et l'opération d'affectation destructive. Il détaille l'aiguillage du flux séquentiel via les branchements conditionnels (SI / ALORS / SINON) fondés sur l'algèbre de Boole, ainsi que l'automatisation des traitements répétitifs via les boucles déterministes (POUR) et conditionnelles (TANT QUE). L'objectif pédagogique est d'acquérir une méthode rigoureuse pour concevoir des algorithmes optimisés, de maîtriser l'écriture de code en C++ selon les normes de l'IUT, et d'éviter les pièges critiques tels que les boucles infinies.

1. Les Variables, le Typage et l'Allocation en Mémoire RAM

En programmation impérative, le processeur exécute des instructions en manipulant des données conservées temporairement dans la mémoire vive (RAM). Pour éviter au développeur de manipuler des adresses hexadécimales brutes et complexes (ex: 0x7FFF5FBFF058), l'algorithmique introduit l'abstraction de la variable. Toute variable doit être explicitement déclarée avant son utilisation. La déclaration informe l'ordinateur qu'il doit réserver un espace mémoire spécifique en lui attribuant un identificateur (son nom) et un type. Le type détermine l'espace physique réservé : un entier standard requiert généralement 4 octets (32 bits), tandis qu'un nombre réel à virgule flottante requiert 8 octets (64 bits). L'opération d'affectation respecte un ordre chronologique strict : 1. L'expression située à droite de l'opérateur est intégralement calculée par le processeur ; 2. Le résultat est stocké dans la case mémoire associée à la variable située à gauche. Cette écriture est dite destructive car la valeur qui s'y trouvait préalablement est définitivement effacée et irrécupérable.

- Variable : Emplacement mémoire nommé permettant de conserver une valeur modifiable au cours de l'exécution.
- Typage : Spécification explicite (Entier, Réel, Chaîne, Booléen) fixant la taille d'allocation en octets et les opérations valides.
- Affectation destructive : Opération séquentielle en deux phases (évaluation du membre de droite puis écrasement de la valeur dans le membre de gauche).

Cas concret enseignant : *Le professeur compare la mémoire RAM à un grand meuble à tiroirs : déclarer la variable 'age : Entier', c'est coller une étiquette 'age' sur un tiroir spécifique et fabriquer physiquement ce tiroir à la dimension exacte pour recevoir uniquement un nombre entier. Placer une nouvelle valeur dans ce tiroir chasse et détruit instantanément la valeur précédente.*

2. Les Structures Conditionnelles et l'Algèbre de Boole

Par défaut, un programme informatique s'exécute de façon linéaire et séquentielle, ligne par ligne. Les structures conditionnelles rompent cette séquentualité en offrant des branchements d'exécution alternatifs selon le résultat d'une évaluation booléenne. Le processeur évalue d'abord la condition : si elle renvoie VRAI, les instructions du bloc 'ALORS' sont exécutées et le bloc 'SINON' est ignoré. Si la condition renvoie FAUX, le processeur saute directement au bloc 'SINON' (s'il existe) ou poursuit la suite du programme. En partiel, une attention particulière doit être apportée à l'exhaustivité des cas pour s'assurer qu'aucun état limite du système ne reste non géré.

- Condition Booléenne : Expression logique s'évaluant exclusivement à VRAI (True) ou FAUX (False).
- Bloc SI / ALORS / SINON : Structure de contrôle permettant d'aiguiller le flux d'instructions selon le résultat d'un test booléen.
- Opérateurs Logiques : Connecteurs (ET, OU, NON) permettant de combiner plusieurs conditions simples afin d'exprimer des règles de décision complexes.

Cas concret enseignant : *Exemple du contrôle d'accès au parking du campus d'Anglet / Bayonne : Si (carte_etudiant_valide == VRAI ET parking_complet == FAUX) ALORS lever la barrière ; SINON afficher le message d'erreur 'Accès Refusé' sur l'écran.*

3. Les Structures Répétitives (Boucles POUR et TANT QUE)

Les structures répétitives ou itératives permettent de répéter un même bloc d'instructions sans dupliquer le code source. Le choix entre une boucle POUR et une boucle TANT QUE relève d'une décision d'architecture algorithmique fondamentale. La boucle POUR est préconisée lorsque le nombre d'itérations est déterminé au départ (par exemple parcourir les 500 étudiants d'une promotion). La variable de contrôle s'incrémente automatiquement à chaque passage. La boucle TANT QUE est utilisée lorsque la répétition dépend d'une condition dynamique (par exemple attendre une saisie valide). La condition est évaluée AVANT chaque passage : si elle est fausse dès le premier test, la boucle s'exécute 0 fois. Règle d'or absolue d'ingénierie : dans un TANT QUE, au moins une variable engagée dans la condition de test doit obligatoirement être modifiée à l'intérieur du corps de la boucle pour garantir l'arrêt du programme.

- Boucle POUR (FOR) : Structure itérative déterministe avec compteur automatique, réservée aux cas où le nombre d'itérations est fixe et connu avant le début de la boucle.
- Boucle TANT QUE (WHILE) : Structure itérative conditionnelle, exécutée tant qu'une condition logique reste vraie, adaptée aux répétitions indéterminées.
- Boucle Infinie : Dysfonctionnement critique survenant lorsque la condition de sortie d'une boucle TANT QUE ne devient jamais fausse, bloquant indéfiniment le programme.

Cas concret enseignant : *Le bug du distributeur automatique de billets (DAB) : si l'algorithme oublie de remettre la variable 'carte_inseree' à FAUX après l'éjection physique de la carte, la boucle TANT QUE demandant la saisie du code PIN tourne indéfiniment et bloque l'appareil.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi doit-on obligatoirement spécifier le type d'une variable dans des langages comme C ou C++ alors que ce n'est pas nécessaire en Python ?
Réponse du professeur : En C/C++, la compilation génère du code machine bas niveau qui nécessite de connaître précisément la taille exacte de l'espace mémoire à réserver en octets dans la RAM. En Python, l'interpréteur gère le typage dynamiquement à l'exécution, ce qui simplifie l'écriture mais consomme davantage de mémoire et de temps processeur.

Question étudiant : Quelle est la différence exacte entre l'opérateur '=' et l'opérateur '==' ?
Réponse du professeur : L'opérateur '=' désigne l'affectation (on évalue la droite et on stocke dans la gauche). L'opérateur '==' est un opérateur de comparaison logique qui teste l'égalité entre deux valeurs sans modifier la mémoire.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme C++ complet respectant les normes de l'IUT d'Anglet qui demande 5 notes (sur 20) à l'utilisateur, calcule leur moyenne arithmétique, puis affiche 'Admis' si la moyenne est supérieure ou égale à 10.0, ou 'Ajourné' dans le cas contraire.

Démarche de résolution :
1. Structuration du fichier C++ avec cartouche complet, tripartition (CONSTANTES, VARIABLES, TRAITEMENTS) et indentation Allman. 2. Définition des constantes NB_NOTES = 5 et SEUIL_ADMISSION = 10.0. 3. Initialisation de l'accumulateur 'sommeNotes' à 0.0 avant d'entrer dans la boucle. 4. Utilisation d'une boucle POUR (for) de 1 à 5 car le nombre de répétitions est déterministe. 5. Lecture de chaque note via le flux 'cin' et ajout immédiat dans 'sommeNotes'. 6. Calcul de la moyenne par division réelle par la constante NB_NOTES. 7. Application du test conditionnel 'if (moyenneCalculee >= SEUIL_ADMISSION)' pour déterminer l'affichage final.

```cpp
/**
 * @file moyenne_notes.cpp
 * @author Matéi de Saint Leger (BUT 1 Informatique)
 * @brief Calcul de moyenne de 5 notes avec test d'admission
 * @date 11/11/2026
 */

#include <iostream>

using namespace std;

int main()
{
    // CONSTANTES
    const int NB_NOTES = 5;
    const double SEUIL_ADMISSION = 10.0;

    // VARIABLES
    double sommeNotes = 0.0;     // (clavier) >> accumulateur des notes
    double noteCourante = 0.0;   // (clavier) >> note saisie à chaque tour
    double moyenneCalculee = 0.0;// (calcul) >> résultat de la division
    int i = 0;                   // (compteur) >> variable de contrôle de boucle

    // TRAITEMENTS
    // (clavier) >> saisie des notes via une boucle POUR déterministe
    for (i = 1; i <= NB_NOTES; i++)
    {
        cout << "Entrez la note numero " << i << " (sur 20) : ";
        cin >> noteCourante;
        sommeNotes = sommeNotes + noteCourante;
    }

    // Calcul de la moyenne arithmétique
    moyenneCalculee = sommeNotes / NB_NOTES;

    // Test conditionnel d'admission
    if (moyenneCalculee >= SEUIL_ADMISSION)
    {
        cout << "Resultat : Admis avec une moyenne de " << moyenneCalculee << " / 20." << endl;
    }
    else
    {
        cout << "Resultat : Ajourne avec une moyenne de " << moyenneCalculee << " / 20." << endl;
    }

    return 0;
}
```

Résultat : Le programme exécute exactement 5 itérations de façon séquentielle, cumule les saisies utilisateur de manière déterministe sans réinitialisation intempestive de la somme, effectue un calcul précis en virgule flottante et délivre le statut exact d'admission.

■ Auto-Évaluation Flash d'Examen

Q : Que se passe-t-il lors de l'évaluation de l'ancienne valeur lors d'une affectation destructive ?

R : L'ancienne valeur contenue dans la variable située à gauche est définitivement écrasée et irrécupérable dès que le résultat du membre de droite y est inscrit.

Q : Quelle est la condition indispensable pour qu'une boucle TANT QUE ne devienne pas infinie ?

R : Au moins une variable engagée dans la condition de test du TANT QUE doit obligatoirement être modifiée à l'intérieur du corps de la boucle pour que la condition devienne fausse à un moment donné.

Q : Pourquoi privilégie-t-on une boucle POUR plutôt qu'un TANT QUE pour parcourir un tableau ou un ensemble de taille fixe ?

R : La boucle POUR est déterministe et gère automatiquement l'incrémentation du compteur, ce qui élimine le risque d'oubli de mise à jour et rend le code plus lisible et sécurisé.